

## SIT225 5.2C

Q1. Describe your method to achieve smooth graphs update in Plotly Dash on arrival of fresh accelerometer data from your phone. Be descriptive enough and use visuals such as flow diagrams and tables, if required.

To make the phone accelerometer graph update smooth without lag or changing chart, the code uses threading and RAM buffers.

How program work:

1. **Phone Sends Data:** phone measures its movement (X, Y, and Z values). It sends these numbers to the Arduino Cloud using the internet.
2. **Python Receives Data:** A background worker in main\_app.py (start\_arduino\_client()) listens for new numbers. When new numbers arrive, it triggers on\_axis\_update().
3. **Saved in Computer RAM:** The numbers are saved directly into computer RAM (deque) instead of reading a slow file from the hard drive. A memory lock (data\_lock) keeps the data safe while it is being saved.

The reason I read from RAM is I want the chart to update in real time. But I found out the reason causing the delay is phone or Arduino app is delay uploading data. So it will also work the same if I read from CSV.

4. **Dash Callback (update\_graph\_realtime()):** On the main thread, a dcc.Interval component triggers the Dash update callback every 100 milliseconds. The callback safely acquires the memory lock, reads the latest data point from the RAM buffer, and returns a lightweight extendData dictionary.
5. **WebGL Canvas Rendering:** The browser receives the extendData payload and appends the new values directly to the active Plotly WebGL line canvas. This avoids redrawing the entire web page, maintaining zero lag and smooth

visual updates.

Explanation:

**Background Threading:** `threading.Thread` runs the `start_arduino_client()` function in the background. This keeps listening for phone sensor data without blocking or slowing down the Dash web server.

**Fast RAM Storage:** Incoming data is pushed into thread-safe deque memory buffers (`time_buffer`, `x_buffer`, `y_buffer`, `z_buffer`) guarded by `threading.Lock()`. Storing values in RAM avoids the lag of reading files from disk continuously.

**Sliding Window:** Setting `maxlen=100` keeps only the latest 100 readings in memory. This prevents browser memory from overloading.

**No UI Jitter:** Adding `uirevision='constant'` keeps zoom levels and axis boundaries stable during updates.

Q2. Describe your new wrapper function that you plan to release for Plotly Dash users to achieve smooth and real-time graphs update without knowing much about the complex data handling. This description should work as a documentation of your wrapper function.

I made a “`create_smooth_dash_app()`” function which is a ready-to-use function designed to quickly launch high-speed Plotly Dash dashboards. It hides complex background mechanics (callbacks, threading memory locks, and WebGL trace updates) so developers can display live time-series data easily.

| Parameter                       | Type             | Default | Description                                                                     |
|---------------------------------|------------------|---------|---------------------------------------------------------------------------------|
| <code>update_interval_ms</code> | <code>int</code> | 100     | The refresh timer interval in milliseconds (e.g., 100 = 10 updates per second). |
| <code>max_points</code>         | <code>int</code> | 100     | Maximum number of continuous data points                                        |

|  |  |  |                                        |
|--|--|--|----------------------------------------|
|  |  |  | displayed on the graph sliding window. |
|--|--|--|----------------------------------------|

- 1. Initializes Dashboard Layout:** Sets up the main page layout
- 2. Pre-builds Line Traces:** Configures three distinct color-coded Scatter lines (X = Red, Y = Green, Z = Blue)
- 3. Automates Real-Time Callback:** Listens to dcc.Interval ticks and fetches the latest values safely out of RAM memory using data\_lock.
- 4. Returns App Instance:** Returns a complete Dash object ready for deployment.

E.G.

```
from main_app import create_smooth_dash_app
```

```
# Create app instance with custom 50ms update rate
```

```
app = create_smooth_dash_app(update_interval_ms=50, max_points=100)
```

```
if __name__ == "__main__":
```

```
    app.run(debug=False, port=8050)
```

Q3. Create a video in Panopto/CloudDeakin showing your program execution, continuous graph updates, your new function working and share the video link here.

<https://youtu.be/td-EEwpWgno>

[https://github.com/SleepinEgg/SIT225\\_2026T2](https://github.com/SleepinEgg/SIT225_2026T2)